

运输层位于面向通信部分的最高层、用户功能的最低层，路由器的实现与使用不涉及运输层
运输层为应用进程间的通信提供服务，即提供应用进程间的逻辑通信

(网络层为主机间的通信提供服务)

运输层向高层用户屏蔽了下层网络核心细节，让应用进程仿佛好像有一条端到端的逻辑通信信道

可靠信道：使用面向连接的协议(TCP) 全双工可靠信道
不可靠：--无连接协议(UDP)

两个主要协议 { 用户数据报协议 UDP — 传送的数据单位协议为 UDP报文 / 用户数据报
传输控制协议 TCP — TCP报文

两者区别：
UDP 可靠的、面向连接的运输服务
传送数据不用先建立连接
收到UDP报文后不发送确认
不可靠交付，但是是一种最有效的工作方式
不提供广播/多播
开销大

运输层的端口

层协议

复用：发送方不同应用进程的发送数据可通过同一运输层传到IP层
解复用：运输层从IP层收到的数据，必须分别交付给指定的进程

如何指明进程？→ 进程标识符PID

应用进程的创建、撤消或改换是动态的，发送方并不了解其他主机的进程⇒不能指定进程作为通信终点

解决→ 端口号 将其设为通信的抽象终点

软件端口是协议栈层间的抽象的协议端口，是应用层各协议进程与运输实体进行层间交互的地点，有别于硬件端口

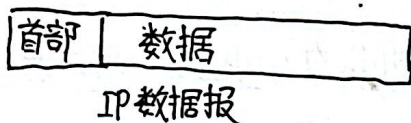
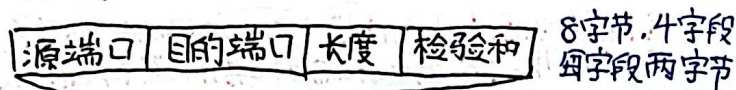
TCP/IP运输层端口用一个16位数字标记，即有65535个端口号，标志本计算机应用层中各进程 不同主机间进程通信，不仅要知道IP，还要有端口号

用户数据报协议UDP

在IP数据报的基础上加了复分和分用、差错检测功能

- 主要特点：
- ① 无连接 发送数据前不用建立连接
 - ② 尽最大努力交付 不保证可靠交付
 - ③ 面向报文 UDP一次传送和交付一个完整报文 (应用层给多少, 就加上首部后直接给IP层) (应用层交下来多少, 加上首部后直接给IP层) (IP层交过来多少, 去掉首部后就全给应用层) → 报文长度应由应用层控制, 太长/短都不好
 - ④ 没有拥塞控制 当网络拥塞时, 源主机发送效率不变
 - ⑤ 支持1对1, 1对多, 多对1, 多对多交互通信
 - ⑥ 首部小, 仅8字节

UDP首部格式



源端口: 需要对应回应时使用, 否则全0

目的端口: 终点交付报文, 必须要有

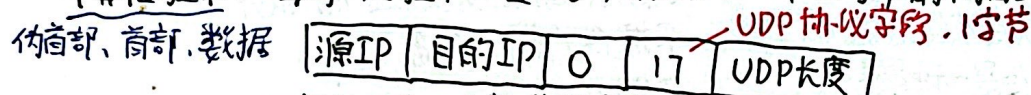
长度: UDP数据报长度, 最小值为8, 单位字节

即仅有首部情况

检验和: 检验UDP数据报是否出错, 错 → 弃

接收方UDP根据首部中的目的端口号交付给进程, 若目的端口不存在, 则丢弃报文并发送ICMP差错报文

计算检验和: 每将“检验和”置全0, 然后加上一个12字节的伪首部一起计算



以每16位为一个数进行求和, 不足16位补0, 然后将求和结果取反, 即为检验和

TCP主要特点

① 是面向连接的传输层协议, 在无连接、不可靠的IP服务上提供可靠交付服务

② 使用前要先建立TCP连接, 使用完释放

③ 一条TCP连接只能有2个端点, 只能是点对点的

④ TCP提供可靠交付服务, 无差错、不丢失, 不重复, 按序到达

⑤ 面向字节流

TCP不保证收到的数据块与发送方发出的具有对应大小关系

但收到的字节流必须与发出的字节流一样

TCP不关心一次发出多长的数据报, 会根据对方给的窗口值和网络拥塞状况来决定

TCP连接的两端是什么? 套接字

套接字 socket = (IP地址: 端口号) 如 192.168.0.1:8000

一条TCP连接唯一地被两端的socket确定

同一个IP地址可有多不同的TCP连接, 多个端口

端口号可出现在不同的...

可靠传输的工作原理

实际网络往往不具备理想的传输条件 $\rightarrow$ 使用协议

停止等待协议

1. 无差错情况 发送方发完一个分组后便停止发送, 等待对方确认后再发送 $P_{2.2}$

2. 出现差错 差错情况 $\left\{ \begin{array}{l} \text{① 接收方检测到 } M_1 \text{ 出错, 将之丢弃} \\ \text{② } M_1 \text{ 在传送过程中丢失} \end{array} \right. \Rightarrow \text{接收方什么也不做}$

发送方如何确认对方收到? **超时重传**

发送方为每个发送分组设一个 **超时计时器**, 若在计时器到期前收到分组 $\rightarrow$ OK

没收到 $\rightarrow$ 重发分组

注: 1. 发完分组后应保留副本 2. 对分组和确认分组的编号

3. 超时计时器重传时间 $>$ 平均往返时间

3. 确认丢失和确认迟到

接收方发送的确认分组丢失/迟到, 发送方重发分组

接收方收到重复分组后, 丢弃并发送确认报文

以上机制可以保证可靠通信, 也叫自动重传请求 ARQ. 但是 **信道利用率太低**

信道利用率 $U = \frac{T_D}{T_D + RRT + T_A}$ T_D : 发送分组所用时间 RRT : 往返时间 T_A : 发送确认所用时间

改进 $\rightarrow$ **流水线传输**: 收到确认以前便继续发送 $P_{2.4}$

连续 ARQ 协议

发送窗口: 发送方维持一个发送窗口, 窗口内分组可以被连续发送出去 $P_{2.4}$

... 滑动: 发送方每收到一个确认, 便将窗口向前滑动一定位置

累计确认: 接收方对按序到达的最后一个分组发送确认, 表示: 包括该分组在内之前所有分组都正确收到

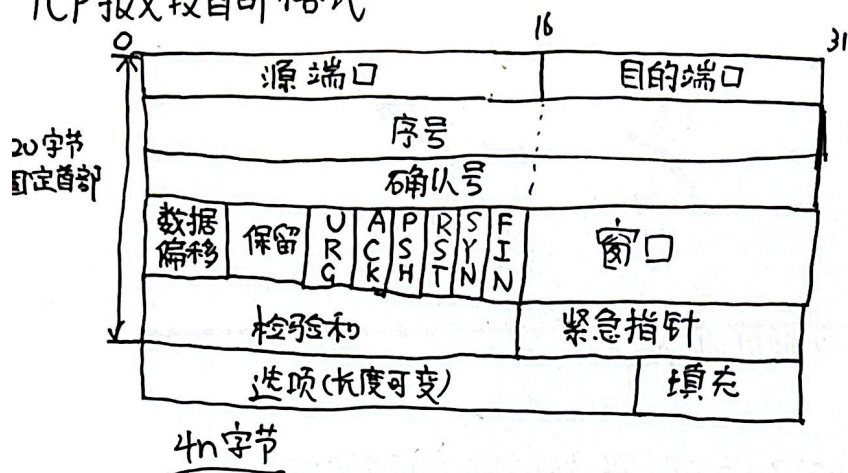
└ 优: 容易实现, 确认丢失也不用重传

缺: 不能反映接收方已正确收到的所有分组信息

回退 N 机制: 退回去重传已发送的 N 个分组

通信质量不好时, ARQ 会负面影响

TCP报文段首部格式



TCP面向字节流,但传的仍为报文段

TCP报文分为首部、数据两部分

① 源、目的端口: 2字节, 运输层的复用、分用通过端口实现

② 序号: 占4字节

TCP连接的字节流中每个字节都有一个序号

序号字段的值即为本报文数据中第一字节的序号

③ 确认号: 4字节

期待收到对方的下一个报文段的数据的第一字节的序号

确认号为N, 则说明序号N-1为止所有数据都收到

④ 数据偏移: 4位 TCP报文数据部分距离TCP报文起始处的距离

单位为4字节(32位), 即表示首部不超过60字节, 选项字段不超过40字节

⑤ 保留: 6位, 全为0, 以后使用

如: ctrl c

⑥ URG 紧急: 控制位 URG=1, 表明紧急指针有效, 该报文段数据重要, 愿不排队先传送

⑦ ACK 确认: 控制位 ACK=1, 确认号字段才有效

⑧ PSH 推送: 控制位 接收到PSH=1的报文后, 优先交给进程, 不等缓存填满

⑨ RST 复位: --- RST=1, 说明TCP连接严重出错, 必须释放连接并重新建立

⑩ SYN同步: --- SYN=1说明这是一个连接请求/连接接受报文

{ SYN=1, ACK=0 连接请求
SYN=1, ACK=1 接受

⑪ FIN终止: --- 要求释放TCP连接

⑫ 窗口: 2字节 从本报文段首部的确认号算起, 接收方目前允许对方发送的数据量

窗口值经常动态变化

⑬ 校验和: 2字节 检验首部和数据, 计算校验和时加上12字节伪首部

伪首部中协议字段为6, 而UDP的为17

⑭ 紧急指针: 2字节 URG=1时有效 指出数据中紧急数据在报文段的末尾位置 (紧急数据之后为普通数据)

⑮ 选项: 一些选项如 MSS 最大报文段长度 → TCP报文中数据字段最大长度

默认最大为536, 因为20+20+536=576

TCP首 IP首 IP长度

窗口扩大, 时间戳
3字节 10字节

TCP可靠传输的实现

以字节为单位的滑动窗口

TCP使用流水线和窗口协议实现高效可靠传输

发送方和接收方分别维护一个**发送窗口**和一个**接收窗口**

发送窗口: 未收到确认的情况下, 发送方可**连续将窗口内数据发送出去**, 发送过的数据暂存

接收窗口: **只允许接收落入窗口的数据**

△P30-233

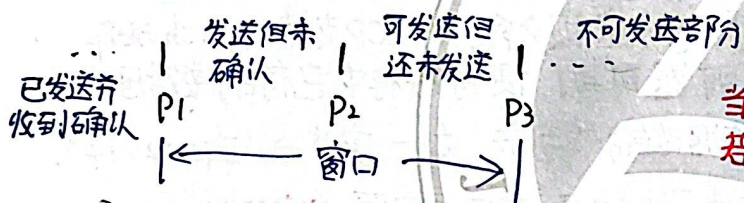
凡是发送过的数据, 在未收到确认时都暂存, 以便超时重传使用

发送窗口内的字节序号表示可以发送的序号, 窗口个, 连续发送数据个, 传输效率高

发送窗口大小不超过接收窗口, 同时还会受网络拥塞影响

P30 发送窗口前边与后边的变化

描述发送窗口用三个指针: P_1 P_2 P_3



$P_3 \sim P_2$ 表示可用/有效窗口

当 P_2 与 P_3 重合, 窗口内都已发送完

若经过一段时间(超时计时器)还未收到确认 → 重传

P32 发送缓存与接收缓存

发送缓存存放: ① 应用进程传来的准备发送的数据 ② TCP已发送未确认数据

接收: ...: ① 按序到达且未被进程读取的数据 ② 未按序到达数据 (若进程不及时读取, 接收缓存会被填满)

PS: ① 发送窗口 ≠ 接收窗口大小

② 对于不按序到达的数据, 先暂存在接收窗口, 待所缺少的字节收到后再交付

例 $\boxed{31}$ $\boxed{32}$ $\boxed{33}$

↓
未达

△
已到达

先将32, 33暂存, 发送给发送方的确认报文中确认号仍为31

③ 接收方有**累积确认的功能**, 接收方在合适时候发送确认报文, 但不应不久

TCP的流量控制

利用滑动窗口实现流量控制

一般来说,希望数据传输快,但接收方可能会来不及接收,造成数据丢失

→ 流量控制: 让发送方发送速率别太快,让接收方来得及接收 → 滑动窗口机制

B₃₆ 在建立连接时,设发送方为A,接收方为B

B告诉A: "接收窗口为X字节"

随后A向B发送数据, B向A发送确认,同时告诉A还能发送多少数据(窗口大小)

当B的接收窗口满了后,便让A不再发送。

死锁? 当B又有了存储空间,向A发送信息告知可以继续发送

但若该报文丢失? → 持续计时器

只要TCP连接一方接到零窗口通知,则启动持续计时器

当时间到了时,发送一个零窗口探测报文段(含1字节数据),接收方确认该报文并给出当前窗口值

若窗口不为0,则可以收到确认与 rwnd

否则收到确认报文方重设持续计时器

TCP的传输效率

进程将数据发给发送缓存后,剩下任务由TCP完成或

TCP发送报文有3个机制: ① TCP维持一个变量=最大报文段长度MSS,当缓存中数据达到MSS便组合成一个报文并发送

② 由应用进程指明要求发送报文段,即TCP支持的push操作

③ 当发送方的一个计时器到期,便将缓存中已有的数据发送

→ 如何确定发送时机? 数据太少,则有效数据传输效率低. 如1字节 → $\frac{1}{41} = 2.44\%$

Nagle 算法 对于应用进程所要发的数据,先写入缓存,然后发送第1字节数据

后面的字节先存起来,当收到第一个字节的确认后,再将缓存中所有数据发送

↓
针对发送小数据包 只有收到前一个确认,才发送下一个报文

还规定: 当到达的数据达到发送窗口大小的一半,或达到最大报文段长度MSS便立即发送一个报文段

接收方糊涂窗口综合症: 接收方的进程读取缓存太慢,

→ 解决: 让接收方B等待一段时间,使得接收缓存足够容量一个最长报文段
或 接收缓存有一半空闲空间时,才发送确认报文,并告知当前窗口大小

江南大学

TCP拥塞控制

拥塞: 网络中对某资源需求 $>$ 可用资源, 此时网络性能变坏, 吞吐量随输入负荷变大而急剧下降
产生原因: 节点缓存容量小、链路容量不足、处理机效率低等 增加资源并不能改善拥塞

流量控制: 抑制发送方发送速率, 使接收端来得及接收
拥塞控制: 防止更多数据进入网络, 避免路由器、链路过载

B40 拥塞控制作用

拥塞控制前提是网络能承受现有的负荷

开环控制: 设计网络时事先考虑周全, 力争避免拥塞, 一旦开始运行便不中途改正
闭环控制: 基于反馈环路概念, 在发生拥塞后, 根据当前网络状态采取措施 $\rightarrow$ P240

TCP拥塞控制方法

四种算法: 慢开始、拥塞避免、快重传、快恢复

TCP用基于滑动窗口的方法进行拥塞控制 (闭环控制)

发送方维持一个拥塞窗口 $cwnd$ 其大小取决于网络拥塞程度且为动态变化

真正的发送窗口值: $\min(\text{接收方通知的窗口值}, \text{拥塞窗口值})$

$cwnd$ 变化原则: 不拥塞时, $cwnd \uparrow$, 发送更多分组; 拥塞时, $cwnd \downarrow$

发送方如何判断拥塞? 超时重传计时器启动时 $\triangle$

1. 慢开始

目的: 探测网络负载能力 / 拥塞程度

过程: 由小 $\rightarrow$ 大逐渐增大注入网络的字节数 / 拥塞窗口数值

每收到一个对新报文段的确认, 增加至多一个发送方最大报文段 $SMSS$

$cwnd$ 每次增加量: $\min(SMSS, h)$ h : 原先未确认, 但现在刚收到的确认报文所确认的字节

首先令 $cwnd = 1$, 发送报文, 收到一个确认, $cwnd = cwnd + 1 = 2$

发送报文 (2个 $SMSS$), 收到2个确认, $cwnd = cwnd + 2 = 4$

... 窗口大小指数增加, 不慢

一个传输轮次经历时间即 RTT 往返时间

传输轮次: 将窗口中所有允许报文段发送, 并收到最后一个报文段确认

为防止 $cwnd$ 增长过大, 还设置慢开始门限 $ssthresh / SSTH$

$\begin{cases} cwnd < ssthresh & \text{使用慢开始} \\ cwnd > ssthresh & \text{停用} \dots, \text{用拥塞避免算法} \\ cwnd = ssthresh & \text{两者都可用} \end{cases}$

若在中断判断出现拥塞 (重传定时器超时)

$ssthresh = \max(\frac{cwnd}{2}, 2)$

$cwnd = 1$ 并执行慢开始, 使发用拥塞的路由器有时间处理和积压分组

2. 拥塞避免

目的: 让 $cwnd$ 缓慢增大, 避免拥塞

过程: 每经过一轮发送与接收确认 RRT, 无论收到多少确认, $cwnd = cwnd + 1$

当发送方判断出现拥塞, 同慢开始, 一样执行 (前一页). 同样令 $ssth = cwnd / 2$

P243 $cwnd$ 变化介绍

3. 快重传

目的: 让发送方尽快知道某个报文丢失 $\rightarrow$ 提高 20% 网络吞吐量

发送方连续收到三个重复确认 3-ACK, 便立即执行“快重传”

P244 $\triangle$ 快重传要求接收方立即发送确认, 即使收到的报文段失序

4. 快恢复

发送方收到 3-ACK, 不执行慢开始, 而是执行“快恢复”

令 $ssthresh = cwnd / 2$, 并令 $cwnd = ssthresh$, 接着执行拥塞避免算法
即 $cwnd = cwnd / 2$

发送窗口上限值 = $\min(rwnd, cwnd)$

$rwnd < cwnd$, 接收方接收能力限制了发送窗口
 $rwnd > cwnd$, 网络拥塞限制了

$cwnd$: 拥塞窗口
 $rwnd$: 接收方窗口

TCP 运输连接管理 $\rightarrow$ 使 TCP 建立与释放正常进行

TCP 连接三个阶段: 连接建立 $\rightarrow$ 数据传送 $\rightarrow$ 连接释放

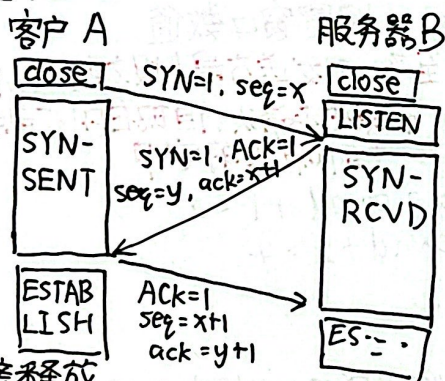
1. 连接建立

要解决的问题: 1. 使每一方都知道对方存在 2. 允许双方协商参数 3. 对运输实体资源分配

TCP 建立连接过程叫握手, 采用三报文握手

建立采用 C/S 方式, 主动发起建立连接的应用进程为客户, 被动等待 - 服务器

P241 三握手过程



首先 A 向 B 发送连接请求报文, 其中 $SYN=1$, 并选择序号 x
 $SYN=1$ 的报文段不携带数据, 但要消耗一个序号

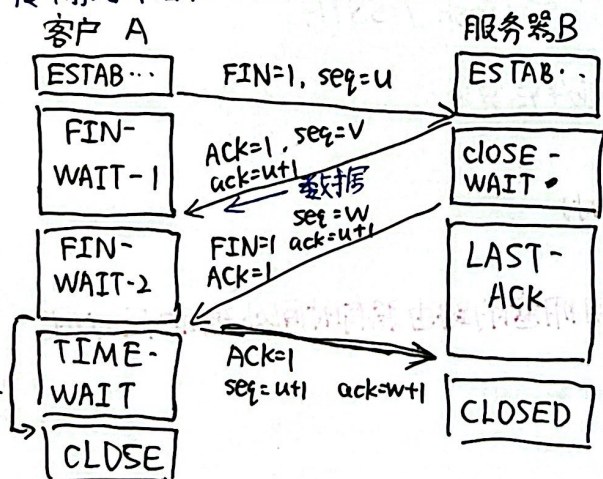
B 收到连接请求报文后, 若同意, 则发回确认

其中 $SYN=1$, $Ack=1$, 确认号 $ack=x+1$, 初始序号 y
同样不携带数据, 且消耗序号

最后 A 再向 B 发出确认, $Ack=1$, 确认号 $ack=y+1$, 序号 $x+1$
如果不携带数据, 则不消耗序号, 此时下一个报文序号为 $x+1$
(再次确认是防止已失效的连接请求报文被 B 收到)

2. 连接释放

传输结束后, 双方都可释放连接, 释放过程采用四报文握手 P249



1. 首先 A 发送连接释放报文, $FIN=1$, 序号 $u = \text{最后一个字节} + 1$
然后 A 进入终止等待 1 不带数据, 消耗序号

2. B 发送确认号 $ack=u+1$, 并进入关闭等待
此时 A 无数据要发, 但若 B 要发, 仍可以发送

3. A 收到确认以后, 并进入终止等待 2
等到 B 发出连接释放报文, $FIN=1$, $ack=u+1$, $seq=w$
B 又发送数据

4. 最后 A 向 B 发送确认, $Ack=1$
 $seq=u+1$, $ack=w+1$, 并等待 2MSL 时间才释放连接

2MSL 由时间等待计时器设置 为什么等?

1. 保证最后一个 ACK 报文到达 B
2. 防止失效的“连接请求报文”出现

MSL: 最长报文寿命
2分钟